

Transformation Rule System

领域架构方案说明

面向银行报文转换场景的、可治理 AI 语义基座

目标读者：Domain Architecture / Platform Architecture / Technology Governance

定位

架构定位

RTS 是一个领域架构能力：它提供受治理的报文转换规则知识基座，并清晰分离规则真相、运行时投影、索引检索和 AI 消费层。

价值

战略价值

RTS 提供可复用、可审计、可扩展的语义层，让多个 AI 能力共用同一套规则真相，而不是各自做孤立的 chatbot 或临时 RAG。

1. 架构摘要

RTS 应被评估为一个领域架构模式，而不是内部知识整理文档，也不是单一 AI use case。它的目标是在银行报文转换场景中建立可治理的语义基座，让 AI 能力基于已审核的规则真相工作，而不是直接依赖原始文档、散落代码或口头解释。

核心
核心判断 通用 RAG 能检索文本，但 RTS 定义的是受治理的规则真相。在银行场景里，字段级转换逻辑、签核状态、歧义和证据边界必须显式存在。
产出
架构产出 一套可复用基座：规则解释、字段血缘、影响分析、发布准备度、客户接入准备度、审计解释等多个应用，都消费同一套受治理的规则层。

推荐表述：

RTS 不是另一个聊天机器人，而是一个受治理的领域知识架构，用于支撑银行报文转换中的规则驱动 AI 能力。

2. 架构问题陈述

当前报文转换知识通常分散在代码、XSLT、mapping Excel、XML 样本、ticket、邮件、支持记录和 Confluence 页面中。如果直接在这些弱治理内容上引入 AI，会产生架构级风险。

架构风险

规则真相分散

不同团队依赖不同材料：代码、Excel、文档、历史 ticket 和 SME 记忆。缺少统一、受治理的规则对象模型。

架构风险

解释能力不足

很难清晰解释某个目标字段为什么生成、依赖哪个源路径、是否经过 review 或 signoff。

架构风险

通用 AI 风险

普通 chatbot 可能给出流畅答案，但不理解产品范围、歧义状态、签核边界和证据要求。

架构风险

复用能力弱

如果每个 AI 场景各自做 prompt 和文档检索，最终会形成重复逻辑和不一致答案。

3. RTS 的架构原则

核心原则是“真相分层”。RTS 将受治理的规则真相、运行时检索和 AI 交互分离。LLM 可以读取、导航和解释，但不能发明业务逻辑，也不能把歧义变成确定结论。

分层
真相层 Canonical pack 定义规则、lookup、helper、证据、review 状态、歧义和 signoff 边界。
分层
投影层 只把已批准、适合运行时使用的规则对象投影给 AI；治理噪声不会被无差别混入上下文。
分层
索引层 投影资源获得稳定 URI、metadata、L0/L1/L2 摘要、依赖关系和检索轨迹。
分层
查询 / AI 层 Agent 通过受 scope 约束的检索和结构化返回消费索引层；需要人工复核的输出仍然只是草稿。

架构不变量：

RTS 决定什么是真的；索引层决定如何找到真相；AI 解释真相，但不创造真相。

4. 参考架构

建议采用分层架构，避免领域治理和 AI 运行时行为混在同一个不可控表面里。

参考
01 受治理的源材料 XML / FpML / SCBML 样本、mapping 表、Java/XSLT 逻辑、lookup 表、defect 历史、review comments 和业务澄清。

05 API 化 AI 服务

通过结构化 prompt 和结构化输出支持规则解释、字段血缘、影响分析、准备度评估和审计叙事。

06 人工复核与流程集成

AI 输出进入 onboarding、UAT、release、incident 或 audit 流程，并保留 evidence link 和人工复核边界。

5. RTS 支撑的 use case landscape

对领域架构而言，重点不是某一个 use case，而是多个 use case 可以共用同一套受治理语义基座。

基座
基座能力 规则解释；字段血缘；Source-to-Target 追踪；规则依赖导航；证据支撑解释；歧义与签核感知回答。
流程
交付与治理流程 规则驱动影响分析；回归测试推荐；发布准备度检查；规则驱动 incident 解释；变更意图校验。
业务
业务侧应用 客户/上游接入加速；新产品/新交易流准备度评估；客户影响分析；监管/审计解释层；组织知识恢复。
扩展
后续扩展 知识过期检测；报文质量雷达；值班交接；隐性依赖发现；运营摩擦分析。

6. 为什么比孤立 AI 案例更强

孤立 AI 案例容易快速 demo，但难治理、难复用、难扩展。RTS 的价值是把多个 use case 变成同一个领域基座的消费者。

收益
复用 一套受治理规则基座支撑多个应用，避免每个 use case 重复做 prompt、检索逻辑和业务假设。
收益
一致性 不同应用从同一套 approved rule objects 回答，减少团队之间解释不一致。
收益
可追溯 答案可以回到规则对象、依赖关系、证据和 review 状态，而不是模糊引用文档片段。
收益
治理能力 歧义和 signoff 状态是一等架构概念，而不是靠 prompt 临时提醒。

可扩展

新 channel、新 product、新 pack 都可以沿用同一模式：真相 → 投影 → 索引 → 受控 AI 消费。

7. 为什么 Copilot / Yoda 不能替代

这里应表达为“产品定位不同”，不是批评工具。
Copilot 和 Yoda 仍然有价值，但它们不提供 RTS 所提出的架构层。

不可替代
Copilot Copilot 是个人生产力助手。它适合写作、总结和辅助个人分析，但不是受治理的报文转换规则真相层，也不是可嵌入流程的后台分析服务。
不可替代
Yoda Yoda 是 Confluence 知识库问答。它适合回答已索引页面的问题，但 RTS 需要处理规则对象、XML 路径、mapping extract、lookup 依赖、signoff 状态和非 Confluence 的操作材料。
需要
RTS + API Token API 化 AI 允许服务级集成：结构化输入、固定输出 schema、prompt 版本管理、validation、证据引用、人工复核和审计日志。

架构区别：

Copilot 和 Yoda 是面向用户的辅助工具；RTS 是受治理的领域架构基座，AI 只是规则真相的受控消费者。

8. 为什么需要 API Token

需要 API token 的原因是：该能力不是人工问答，而是嵌入领域流程的后台 AI 能力，并且受架构规则约束。

Token
流程集成 从 onboarding、UAT、release、incident 或 audit 流程中触发分析，而不是依赖人工聊天会话。
Token
结构化材料 传入 XML 样本、mapping extract、rule object、defect list 和 metadata 等结构化输入。
Token
受控输出 要求固定 JSON、checklist 或 assessment 模板，并支持 validation 和 retry。
Token
审计能力 记录输入来源、prompt 版本、模型版本、输出、复核人和 evidence links。

Token

安全控制

支持脱敏、scope enforcement、访问控制和 human-in-the-loop 审批。

Token

平台扩展

暴露可复用 AI 服务层，让多个 RTS-enabled 应用调用。

9. 架构护栏

这些护栏是 Domain Architecture 审批时最重要的部分，用来证明方案是受治理基座，而不是不可控生成式系统。

护栏

AI 只读消费

AI 读取投影后的规则真相和操作证据，不修改生产规则、mapping、代码或配置。

护栏

人工决策边界

AI 输出是分析草稿、checklist、summary 或 recommendation；最终 signoff 仍由授权 owner 完成。

护栏

Scope 约束

检索必须先解析 channel、product、pack、object 范围，避免跨产品或跨系统污染。

护栏

保留歧义

open question、缺失证据、未签核规则必须保持可见。
Unknown is better than wrong。

护栏

运行时学习不能写回真相

session memory 或用户反馈不能静默改写 canonical rule truth
；更新必须经过 authoring / review / signoff。

护栏

输出可追溯

关键结论应引用 rule object、dependency、evidence 或
review 状态。

10. 建议 Pilot：验证架构，不是展示聊天

Pilot 应验证架构模式，而不是只展示 chatbot。好的 pilot 应证明：受治理规则对象可以安全地支撑多个输出。

Pilot
Pilot 范围 选择一个 source-target channel，例如 Tradition → Stella，并选取 COMMON / FXD / FXO 中少量已批准 packs。
Pilot
基座输出 从投影规则对象中生成规则解释、字段血缘、依赖导航和歧义感知回答。
Pilot
流程输出 针对一个受控样本变更，生成规则驱动影响分析和回归测试推荐。
Pilot
业务输出 结合 sample XML 和现有规则覆盖，生成 onboarding gap checklist 或产品准备度摘要。

架构验收标准

scope 解析正确；规则引用可追溯；歧义被保留；输出结构化；人工复核边界明确；API 调用可重复。

11. 给 Domain Architecture 的决策点

架构评审应关注：是否接受 RTS 作为 AI-enabled message transformation 的语义基座模式，以及需要哪些边界来安全采用。

决策
1. 批准分层架构 Truth Layer → Projection Layer → Index Layer → API-based AI Consumption。
决策
2. 批准 RTS 作为规则真相源 Canonical packs 是 transformation rules 的 court of record。
决策
3. 批准 API 化 AI 作为受控消费者 AI 可以解释、总结、分类和生成草稿，但不能决策或修改真相。
决策
4. 批准 Pilot 范围 从窄 source-target channel 和 selected approved packs 开始验证检索和 workflow outputs。

5. 定义 ownership

Domain owner 负责规则真相；platform owner 负责 index/API；workflow owner 负责消费应用。

12. 可直接放进架构材料的表述

Transformation Rule System 建议定位为面向银行报文转换场景的受治理领域知识架构。它把分散的转换知识整理成经过 review、感知 signoff、可寻址的规则对象，并通过索引检索和 API 化 AI 消费暴露已批准的运行时投影。

该方案的目标不是再建设一个聊天机器人，而是建立一套可复用语义基座，让规则解释、字段血缘、影响分析、测试推荐、发布准备度、客户接入准备度和审计解释等多个 AI 能力可以安全地构建在同一套规则真相上。

Copilot 和 Yoda 仍然适用于个人生产力和 Confluence 知识问答，但它们不能替代受治理的规则真相层、结构化运行时投影、scope 约束检索、证据引用输出和可嵌入流程的 API 服务。

控制声明：

AI 不做生产决策，不批准规则，不修改 mapping，也不改变系统行为。AI 生成的是结构化、可追溯、带证据引用的草稿，供授权人员复核。